



南京大學
NANJING UNIVERSITY

Empirical Software Engineering

Software System Design (2026) - Architecture
Assignment 1

NanJing University

Teacher: 张贺

ID: 231250097 Name: 姚凯方

Date: 2026.5.20

1 任务 1: 质量属性场景

1.1 AI-Enabled Applications vs AI-Native Applications

1.1.1 AI-Enabled Applications

AI-Enabled Applications 的特征是在已有的软件系统中**集成人工智能能力**，以增强功能、提高效率或者改善用户体验，同时**不改变宿主系统的核心行为** [1]。在此类应用中，AI 模块在概念上是可拆卸的：移除它不会影响基础系统执行核心功能的能力 [2]。这种模块化特性类似于面向对象设计中的**开闭原则**，该原则主张软件实体应对扩展开放、对修改封闭 [2]。在 AI-Enabled Applications 中，添加 AI 相当于功能的扩展，而不是对原有实现的修改。由于 AI-Enabled Applications 开发的重点是优化宿主应用中已实现的功能，这类 AI 增强通常具有**有限的可扩展性和适应性**。一个典型案例是 **Salesforce Einstein**，它是在 Salesforce 客户关系管理 (CRM) 平台中嵌入的一组 AI 增强功能 [3]。Salesforce 的核心 CRM 功能——如数据存储、管道管理和工作流自动化——在 Einstein 模块未激活的情况下仍然保持完整。Einstein 提供的预测评分和下一步行动推荐提升了用户的生产力和决策支持，但并未改变原系统的结构性行为或要求 [3]。因此，这类应用虽然能够提升现有流程效率，但其功能和性能的扩展能力通常受限于原有系统架构，难以支持根本性的业务创新或应对快速变化的需求环境。

1.1.2 AI-Native Applications

AI-Native Applications 的特征是 **AI 是功能的核心板块**。AI 作为核心板块就意味着它会优化原有提供服务的逻辑，因此这类软件如果在已有的软件上重新开发，往往需要对整体架构进行**全面调整**；而对于从零开始的新应用开发，也应当以 AI 的核心流程作为系统设计的基础 [4, 5]。由于 AI 是系统的核心，这类应用通常无法脱离 AI，一旦丧失 AI 能力，系统就无法正常提供预期的服务 [4]。同时，由于系统从设计初期就围绕 AI 能力构建，这类应用通常具有较好的扩展性和适应性，能够支持新的业务功能和复杂场景的演化。一个典型的实例是 **OpenAI 的 ChatGPT 平台及其 API 生态**。在 ChatGPT 中，语言生成模型不是简单作为某个附加功能引入，而是系统的核心功能——包括自然语言理解、对话管理、生成响应等均依赖于 AI 模型的实时推理能力。若去除 AI 模块，整个系统将失去主要的服务能力，无法完成问答、文本生成、指令解析等功能。更重要的是，由于 ChatGPT 的基础架构是围绕 AI 模型设计的，它可以非常容易地扩展出新的能力：多模态支持、插件与工具链集成、行业定制版本、上下文记忆、长期对话扩展。[6] 这些都是依赖 **AI 能力本身的扩展**，而不是对原有传统逻辑的增强，这正体现了 AI-Native Applications 比 AI-Enabled Applications 比具有更强的可扩展性和适应性 [4]。

1.1.3 总结

对 AI-Enabled Applications 和 AI-Native Applications 的差异对比总结如下表 1。

特征维度	AI-Enabled Applications	AI-Native Applications
AI 角色	辅助业务流程	核心驱动功能
系统依赖	系统可脱离 AI 运行	离开 AI 无法提供核心功能
设计阶段	可在已有系统后期集成 AI	从设计初期即围绕 AI 构建架构
业务影响	提升效率与性能，优化现有流程	支持创新、颠覆业务模式与新功能
扩展性与适应性	可扩展性有限，适应性弱	高可扩展性，适应新任务和场景能力强
典型案例	Salesforce Einstein [3]	ChatGPT / GPT 平台 [6]

Table 1: AI-Enabled Applications 与 AI-Native Applications 对比

1.2 质量属性

1.2.1 质量属性的选择

针对 AI-Enabled Applications 和 AI-Native Applications，选择了两个关键质量属性，分别是：

- 1. **AI-Enabled Applications: Availability 和 Modularity。**
 - 1) **Availability (可用性)**: 保证 AI 模块在系统运行期间能够及时响应用户请求，增强功能可用而不影响核心业务流程。
 - 2) **Modularity (模块化)**: AI 是附加模块，系统设计需允许其独立升级、替换或卸载而不破坏核心功能。模块化体现了插件式架构和接口解耦，使 AI 功能可灵活扩展或更新的程度。
- 2. **AI-Native Applications: Availability 和 Robustness。**
 - 1) **Availability (可用性)**: AI 是系统核心，必须随时可用，否则核心功能无法提供。
 - 2) **Robustness (鲁棒性)**: 系统在面对异常输入、模型波动、部分组件故障或高负载场景时，仍能够维持稳定、可控且可预测服务行为的能力。

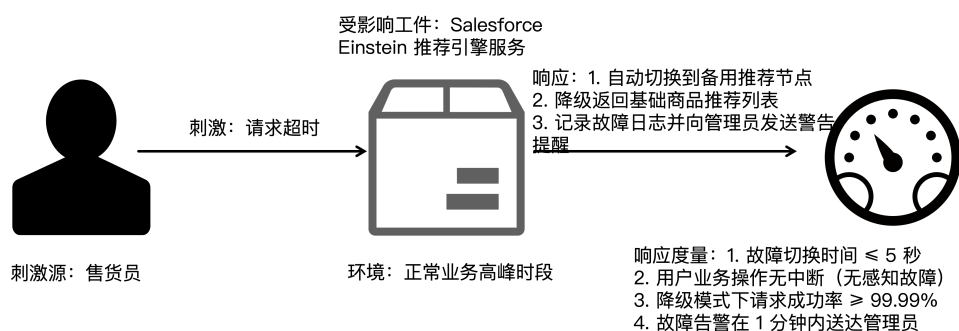
应用类型	质量属性 1	质量属性 2
AI-Enabled Applications	Availability	Modularity
AI-Native Applications	Availability	Robustness

Table 2: AI-Enabled 与 AI-Native Applications 的关键质量属性选择及理由

1.2.2 场景分析

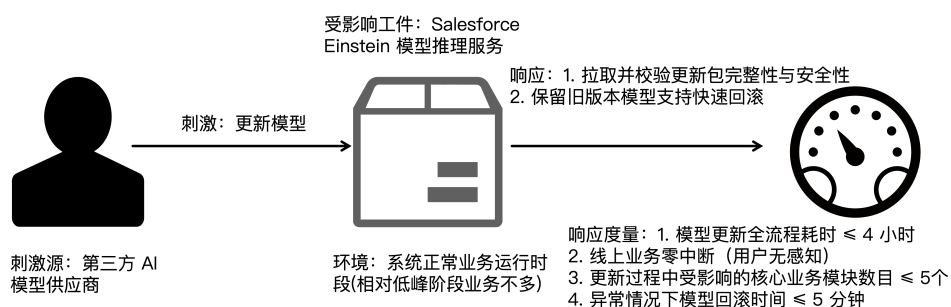
场景组成部分	可能取值
刺激源	普通用户、恶意用户、AI 服务、数据库、网络基础设施、云平台、系统管理员
刺激	高并发请求、AI 推理负载过高、GPU 资源耗尽、AI 服务超时、AI 模块崩溃、数据库连接失败、网络通信故障、云服务不可用、恶意请求攻击
受影响工件	核心业务系统、AI 推荐模块、API 网关、通信通道、模型推理服务、数据库服务、工作流服务、用户接口模块
环境	正常运行、高峰流量、降级运行、AI 模型更新阶段、系统维护阶段、云资源竞争环境
响应	检测系统组件异常 - 记录故障日志与异常推理行为 - 通知监控系统或维护人员 - 隔离异常 AI 模块，防止故障扩散 - 将请求重定向至备用服务实例 - 临时关闭 AI 增强功能或进入降级模式 - 保证核心业务功能继续运行 - 在无 AI 支持情况下继续提供基础服务
响应度量	核心业务系统可用率（例如 99.99%） AI 增强功能可用率（例如 99.5%） 系统故障检测时间 核心业务恢复时间（MTTR） AI 模块故障期间成功完成的核心业务请求比例 系统在降级模式下可持续运行的时间

Table 3: AI-Enabled Applications ——Availability 场景



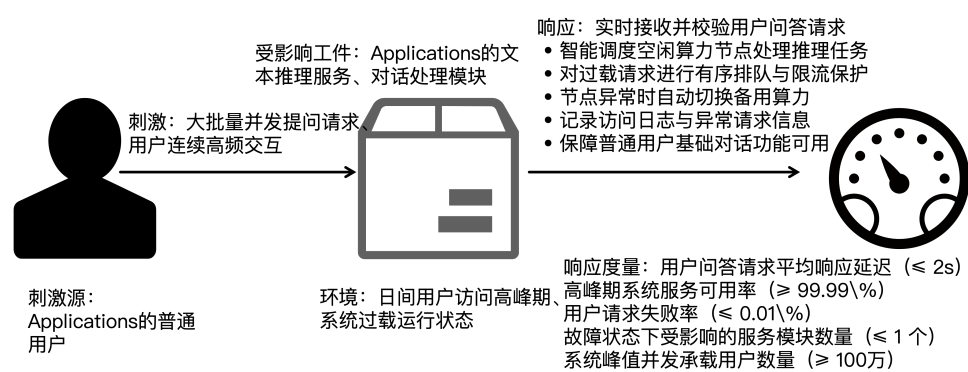
场景组成部分	可能取值
刺激源	开发人员、系统架构师、系统管理员、第三方 AI 服务提供商
刺激	新增 AI 模块、更换 AI 模型、移除 AI 功能、更新接口 API
受影响工件	AI 插件模块、服务接口、中间件、通信 API、微服务组件
环境	正常运行、系统维护阶段、系统升级阶段、持续部署环境
响应	将 AI 模块变更与核心业务逻辑隔离 - 独立替换或升级 AI 模块 - 保持外部接口兼容性 - 保证核心业务功能正常运行 - 支持 AI 模块独立部署与回滚
响应度量	替换 AI 模块所需时间 AI 模块更新影响的核心组件数量 可复用接口/组件比例 AI 模块集成或替换导致的停机时间 部署新 AI 功能所需工作量

Table 4: AI-Enabled Applications —Modularity 场景



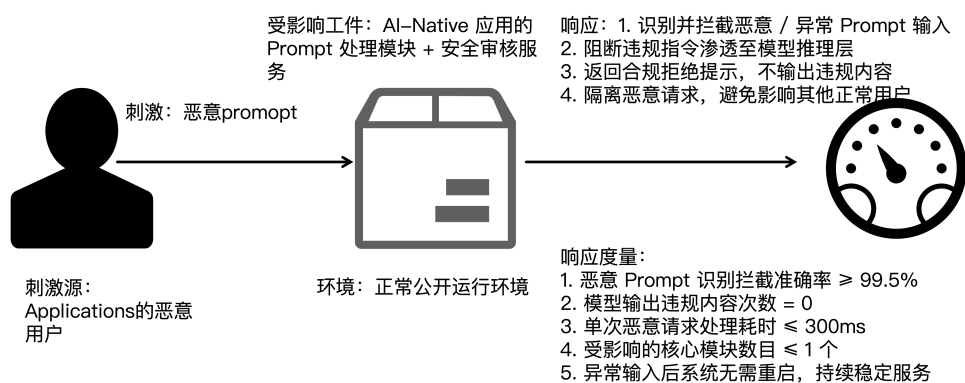
场景组成部分	可能取值
刺激源	用户、AI Agent、分布式推理系统、云基础设施、外部 API 服务
刺激	推理服务中断、GPU 故障、模型服务超时、并发请求过高、分布式服务崩溃
受影响工件	大语言模型服务、分布式推理集群、向量数据库、任务编排系统、API 服务
环境	正常运行、高并发环境、降级运行、部分基础设施失效、模型更新阶段
响应	检测 AI 服务中断 - 记录故障与异常推理行为 - 触发监控告警与自动扩容机制 通过以下方式恢复服务可用性： - 切换至备用推理节点 - 将流量重定向至冗余服务 - 进入降级响应模式 - 临时限制请求吞吐量
响应度量	整体服务可用率（例如 99.99%） AI 服务最大允许停机时间 平均故障恢复时间（MTTR） 高负载下平均响应延迟 降级运行期间成功完成的请求比例

Table 5: AI-Native Applications —Availability 场景



场景组成部分	可能取值
刺激源	用户、对抗性输入、不稳定 AI 模型、异常数据集、恶意 Prompt
刺激	模型幻觉、异常 Prompt、推理不稳定、部分模型失效、输入分布异常变化
受影响工件	核心 AI 模型、Prompt 处理模块、推理流水线、记忆系统、外部工具集成组件
环境	高负载运行、对抗性交互环境、降级运行、不完整上下文环境
响应	检测异常 AI 行为 - 过滤或拒绝恶意/异常输入 - 记录不稳定或不安全输出 - 隔离异常推理组件 通过以下方式维持系统稳定行为： - 返回回退响应 - 限制危险操作 - 切换至更安全或更小的备用模型 - 防止级联故障扩散
响应度量	被安全处理的异常请求比例 降级响应成功率 隔离异常 AI 组件所需时间 被阻止的级联故障数量 系统在异常输入或对抗性环境下的稳定性

Table 6: AI-Native Applications —Robustness 场景



2 任务 2: 策略与战术

2.1 策略战术分析表

质量属性	优化策略	具体战术 (tactics)	收益	负面影响
availability	灰度与滚动升级	流量分级发布与逐步放量; 关键指标监控与自动回滚机制	降低新版本上线导致全量服务中断的风险; 通过小流量验证降低故障传播范围, 减少发布阶段故障率, 提高系统 MTBF; 自动回滚机制可缩短异常恢复时间, 降低 MTTR	灰度流量控制与实时监控增加系统复杂度; 多版本并存可能导致额外资源消耗和响应延迟; 回滚与监控系统增加运维成本与部署时间
availability	多活部署与自动故障切换	多数据中心 active-active 部署; 健康检查驱动的自动故障转移	通过节点冗余与自动故障切换减少单点故障影响; 显著降低 MTTR; 提升 MTBF, 提高整体可用性	多活架构需要额外计算与网络资源; 跨节点状态同步增加延迟; 一致性与故障检测机制提高实现复杂度
modularity	功能解耦与模块隔离	微服务拆分与 API 契约定义; 消息驱动的异步通信机制	降低耦合度, 提高可维护性; 局部故障隔离; 独立部署减少 MTTR, 提高可用性	服务数量增加导致网络调用增加; 一致性与事务复杂度提升; 治理成本增加
modularity	故障隔离机制	熔断机制控制异常调用; 资源与运行环境隔离	防止局部故障扩散为级联故障; 提高 MTBF; 降低故障传播范围并减少 MTTR	响应延迟可能增加; 资源利用率下降; 隔离策略与监控机制增加系统设计与维护复杂度
robustness	自适应推理与鲁棒生成	上下文压缩适配输入限制; 失败请求自动重试机制	在输入异常、上下文超长或资源受限情况下维持稳定输出能力; 提高请求整体成功率; 降低用户端感知延迟并增强系统鲁棒性	上下文压缩可能导致语义信息损失; 重试机制增加平均响应时延与资源消耗; 流式生成可能降低输出一致性与确定性
robustness	资源调度与弹性容错	负载均衡请求分发; 基于负载的弹性扩缩容	提升系统在高并发场景下的吞吐能力与稳定性; 降低因资源瓶颈导致的故障概率, 提高 MTBF; 通过快速恢复机制降低 MTTR	弹性扩缩容可能引入短时性能波动; 冗余资源配置增加基础设施成本; 轻量模型替代可能导致输出质量下降

Table 7: 质量属性优化策略与战术分析表

2.2 在 availability 角度上对比两类应用

针对 availability 维度，本文提出的两种优化策略分别为基于渐进式变更控制的发布可靠性策略与基于多节点冗余的运行时高可用策略。在可用性方面，AI-Enabled Applications 与 AI-Native Applications 在设计思路与实现机制上存在显著差异。

对于 AI-Enabled 应用而言，可用性保障主要依赖于以系统架构为中心的传统高可用设计方法。其核心思想是将 AI 能力作为相对独立的外部服务或功能模块进行封装，通过冗余部署与失败隔离机制确保当 AI 服务发生异常或不可用时，系统仍能够维持核心业务流程的正常运行。因此，该类系统的整体可用性主要由基础服务架构所决定，而 AI 组件通常不构成系统的关键单点依赖。同时，该架构要求 AI 服务的异常必须被严格隔离，以避免对其他业务模块产生连锁影响，从而保证系统整体稳定性。

相比之下，AI-Native 应用将 AI 推理能力深度嵌入系统的核心执行路径，其可用性不仅依赖基础设施层面的高可用设计，还高度依赖模型服务本身的稳定性以及端到端推理链路的连续性。因此，其可用性保障策略进一步扩展为模型级冗余机制、推理失败自动恢复机制，以及基于请求或 token 粒度的熔断与降级策略，以确保在模型波动、资源受限或推理失败的情况下仍能够维持整体服务能力。此外，由于 AI-Native 应用通常具有更高的业务价值密度，其经济收益能够支撑更高成本的模型冗余与推理基础设施建设，使得多模型备份与动态路由等复杂策略具备实际落地条件。

总体而言，AI-Enabled 系统强调“AI 失效不影响系统可用性”，其核心在于通过架构隔离实现系统级稳定性；而 AI-Native 系统则强调“模型能力与系统能力的一体化可用性”，其核心在于通过端到端推理链路的冗余与恢复机制实现整体高可用。因此，后者在可用性保障机制上更加复杂，同时对模型服务与基础设施的协同设计提出了更高要求。

3 任务 3: 设计决策

3.1 职责分配 (Allocation of Responsibilities)

决策 1: 将 AI 能力与核心业务逻辑解耦 (AI-Enabled)

理由: 在 AI-Enabled 应用中，将 AI 能力封装为独立服务模块，与核心业务逻辑解耦，不仅可以确保当 AI 服务发生故障或不可用时系统仍可继续执行基础业务流程，从而提升系统可用性并降低故障传播风险，同时也使 AI 能力具备更强的独立演进能力与可替换性，使其可以在不影响核心业务的情况下进行升级、优化或模型替换，从而提升系统的可维护性与扩展灵活性。

决策 2: 将推理、规划与决策能力交由 AI 智能体承担 (AI-Native)

理由: 在 AI-Native 应用中，系统的核心职责由 AI 智能体承担，包括推理、规划与决策过程。此时，AI 能力不再仅作为外部扩展功能模块存在，而是上升为系统核心业务逻辑的一部分，直接参与并负责关键业务流程的执行与决策。

3.2 协调模型 (Coordination Model)

决策 1: 基于 API 的同步调用与降级协调机制 (AI-Enabled)

理由: AI-Enabled 系统通常采用基于 API 的请求-响应协调方式，将 AI 能力封装为标准化服务接口，使其通过 API 与业务功能模块进行交互，从而实现系统层与 AI 能力层之间的边界隔离与松耦合设计。这种方式不仅便于不同模块之间的独立开发与部署，还使 AI 服务具备良好的可替换性与可扩展性。同时结合降级策略与备用路径设计，在 AI 服务不可用时提供替代执行路径，从而保障系统可用性。

决策 2：基于多智能体的动态任务协调机制（AI-Native）

理由： AI-Native 系统以 AI 能力作为核心能力底座，在此基础上扩展形成多智能体协同机制，通过运行时动态任务分配与多智能体之间的协作完成复杂任务。该架构使系统具备更强的任务分解与组合能力，从而提升系统适应性与鲁棒性，但同时也引入了智能体之间的协调开销与行为不确定性。

3.3 资源管理（Resource Management）

决策 1：AI 服务冗余部署与资源托管机制（AI-Enabled）

理由： 在 AI-Enabled 系统中，AI 能力作为外部扩展能力存在，并不直接构成系统核心业务逻辑。因此，系统对 AI 能力的设计更偏向谨慎引入与可控使用，其资源投入主要用于保障 AI 服务的基础可用性与可替换性。通过部署多个独立 AI 服务实例，可以在一定程度上提升服务稳定性，但在整体策略上仍允许在 AI 服务不可用时直接关闭或绕过 AI 能力，从而保证核心业务流程的持续运行。因此，该类架构中的冗余更多体现为“非关键能力的可选保障”，其目标是在控制成本的前提下进行有限度的可靠性增强。

决策 2：多模型协同与 AI 能力稳定性保障机制（AI-Native）

理由： 在 AI-Native 系统中，AI 能力构成系统的核心执行基础，直接决定系统对用户请求的理解与响应质量。因此，资源投入的重点不在于“是否启用 AI”，而在于保障 AI 能力持续稳定运行。系统通过多模型协同与动态推理调度机制，在不同模型之间进行资源分配与能力补偿，以确保在高负载或复杂任务场景下仍能维持稳定的推理能力与输出质量。因此，该类架构中的冗余设计主要服务于 AI 能力本身的可靠性保障，其目标是提升模型运行稳定性与结果正确性，而非简单的可选能力覆盖。

3.4 绑定时机（Binding Time）

决策 1：运行时 AI 服务选择机制（AI-Enabled）

理由： 在运行时动态选择 AI 服务提供方或模型版本，使系统具备更强的灵活性与容错能力，在服务异常时可快速切换备选方案，从而提高可用性。

决策 2：基于上下文与资源约束的动态模型切换机制（AI-Native）

理由： AI-Native 系统在推理过程中根据上下文复杂度与资源状态动态调整模型选择，从而在性能与效果之间进行平衡，提高系统整体鲁棒性。

3.5 技术选型（Choice of Technology）

决策 1：基于微服务架构的外部 AI 接口集成（AI-Enabled）

理由： AI-Enabled 系统通常采用微服务架构，并通过外部 AI 接口集成能力，使 AI 模块独立部署，从而降低系统耦合度并提高整体可维护性与可用性。

决策 2：分布式推理框架与模型编排系统（AI-Native）

理由： AI-Native 系统采用分布式推理框架与模型编排机制，对多个模型和智能体进行统一调度，从而提升系统扩展性与鲁棒性，但增加基础设施复杂度与成本。

4 冗余机制分析

4.1 主动冗余（Active Redundancy）

定义： 主动冗余是指多个冗余组件同时处于运行状态，并共同对外提供服务，通过并行执行实现系统级容错能力。

优点：主动冗余可以在节点发生故障时实现无感切换，几乎不存在服务中断时间，从而显著降低 MTTR，提升系统可用性。同时，多实例并行处理还能够提升系统整体吞吐能力与负载处理能力。

代价：该机制需要所有冗余节点持续运行并同时对外提供服务，因此在资源层面会带来显著开销，包括 CPU、内存以及网络带宽的持续占用。此外，由于多个实例需要保持状态一致性，在分布式环境下通常需要引入状态同步机制或一致性协议，从而带来额外的通信开销与延迟放大问题。同时，在写入型或有状态服务场景中，还可能产生写放大效应，进一步增加系统整体负载与实现复杂度。

适用场景及理由：

1. 实时交互系统（如大模型问答系统）：对延迟敏感，需要连续可用性
2. 高并发互联网服务（如电商秒杀系统）：需要应对突发流量并保证稳定性
3. 金融级关键系统（如支付与交易系统）：对 SLA 要求极高，不能接受服务中断

4.2 被动冗余（Passive Redundancy）

定义：被动冗余通常采用主备（Primary-Backup）架构，主节点负责正常业务处理，备用节点在正常情况下仅维持待命或低负载运行状态。当主节点发生故障时，系统通过故障检测机制触发备份节点接管服务，从而恢复系统运行能力。

优点：由于备用节点在正常情况下不直接参与业务处理，因此相比主动冗余能够显著降低 CPU、内存以及网络带宽等资源消耗，整体运行成本更低。同时，该机制无需长期维持多节点强一致状态，降低了分布式同步与协调复杂度。在业务负载较稳定或对实时性要求不高的场景下，被动冗余能够以较低成本提供基础容错能力，在可用性与资源利用率之间取得平衡。

代价：由于系统需要先完成故障检测、主备切换以及状态恢复过程，因此在节点失效后会产生一定服务中断时间，使 MTTR 高于主动冗余。此外，备用节点可能长期处于低活跃状态，在切换时需要进行缓存预热、连接恢复或数据同步，从而进一步增加恢复延迟。在有状态服务场景下，如果主备之间存在同步滞后，还可能导致部分运行状态或数据丢失问题。

适用场景及理由：

1. 企业内部管理系统：该类系统通常允许短时间服务中断，更关注资源成本与稳定运行之间的平衡
2. AI 辅助系统（AI 能力非核心路径）：AI 服务故障不会直接导致核心业务不可用，因此无需长期维持高成本主动冗余
3. 离线批处理系统（如日志分析与数据统计任务）：系统对实时响应要求较低，可以接受一定恢复时间

4.3 备份系统（Spare）

定义：备份系统通常处于离线或低资源占用状态，在正常运行期间不直接参与业务处理，仅在主系统发生严重故障、服务完全不可用或灾难性失效时才被启用，用于恢复系统基础运行能力。

优点：由于备份节点无需长期维持在线运行状态，因此其 CPU、内存以及网络资源消耗最低，整体基础设施成本显著低于主动冗余与被动冗余。同时，该机制能够作为系统最后一级容灾保障，在主系统遭遇极端故障、数据中心失效或大规模服务中断时提供恢复能力，从而提升系统整体生存能力与灾难恢复能力（Disaster Recovery Capability）。

代价：由于备份系统通常不参与实时运行，因此在切换过程中需要完成环境初始化、数据恢复、服务重建以及配置同步等操作，导致恢复时间最长，使 MTTR 显著高于主动冗余与被动冗余。此外，备份节点与主系统之间通常采用周期性数据同步机制，在故

障发生时可能存在数据滞后或部分状态丢失问题。在大规模系统中，冷备恢复还可能受到网络传输、数据恢复速度以及依赖服务重建过程的影响，从而进一步增加恢复复杂度与恢复时间。

适用场景及理由：

1. 灾难恢复系统 (Disaster Recovery Site)：用于应对数据中心级别故障或极端灾难场景
2. 非关键业务系统 (如历史数据归档系统)：允许较长时间不可用，更关注低成本运行
3. 实验性或研究型系统：系统失败不会直接影响核心业务，因此无需高成本高可用架构

4.4 选型标准与总结

三种冗余机制的选择本质上是在可用性、成本与恢复时间 (MTTR) 之间进行权衡，其决策依据如下：

1. 当系统对可用性要求极高且 MTTR 要求极低时，应选择主动冗余
2. 当系统需要在成本与可靠性之间折中时，应选择被动冗余
3. 当系统对成本敏感且可接受较长恢复时间时，应选择备份系统

总结：冗余策略选择的核心是根据业务关键性与 SLA 要求，在资源成本与系统可用性之间进行平衡，从而确定合适的容错架构设计。

References

- [1] T. H. Davenport and D. Ronanki, “Artificial intelligence for the real world,” *Harvard Business Review*, vol. 96, no. 1, pp. 108–116, 2018. [Online]. Available: <https://hbr.org/2018/01/artificial-intelligence-for-the-real-world>
- [2] B. Meyer, *Object-Oriented Software Construction*. Prentice Hall, 1997.
- [3] Salesforce Inc., “Salesforce einstein overview,” <https://www.salesforce.com/products/einstein/overview>, 2025, accessed 2026-05-15.
- [4] L. Cao, S. Li, Y. Fan, D. Li, and C. Zhong, “Towards the next generation of software: Insights from grey literature on ai-native applications,” *arXiv preprint arXiv:2509.13144*, 2025, aI-Native 应用定义与质量属性.
- [5] Z. Wang, G. Yu, and M. R. Lyu, “Ai-nativebench: An open-source white-box agentic benchmark suite for ai-native systems,” *arXiv preprint arXiv:2601.09393*, 2026, aI-Native 架构与工程分析.
- [6] OpenAI, “Chatgpt: Optimizing language models for dialogue,” <https://openai.com/blog/chatgpt>, 2023, accessed 2026-05-15.